

BALANCED LIFE (BLNC)

App Architecture Guide

Repo Structure, Tech Stack, Firebase Data Model & Implementation Steps

Version 1.0 | March 2026

PULSE Team | TechLauncher 2026 S1

1. Technology Stack

The following technologies were selected for the BLNC app:

Layer	Technology	Why
Framework	Expo SDK 53+ (TypeScript)	Managed workflow, test on phone via Expo Go
Navigation	React Navigation	Industry standard, tab + stack support
Client State	Zustand	65% less boilerplate than Redux, 1KB, no Provider wrapping
Server State	TanStack Query (React Query)	Caching, background refetch, loading/error states for Firestore
Auth	Firebase Auth (JS SDK)	Email + Google/Apple sign-in, works in Expo Go
Database	Firebase Firestore	NoSQL, real-time listeners, offline support, free tier
Cloud Functions	Firebase Cloud Functions v2	Server-side score recalculation, scheduled tasks
Charts	react-native-gifted-charts	Radar, line, bar, pie charts in one library
Forms	React Hook Form + Zod	Lightweight form validation, schema-based
Styling	NativeWind (Tailwind CSS for RN)	Utility-first, consistent design system
Notifications	expo-notifications	Abstracts FCM/APNs, simpler than raw Firebase
Animations	react-native-reanimated	Native-thread animations for score reveals
Date/Time	date-fns	Lightweight date manipulation for rolling averages
Testing	Jest + RN Testing Library	Unit + component testing
Linting	ESLint + Prettier	Code consistency across team

2. Complete Repository Structure

The app uses feature-based organisation. The `/app` directory handles routing (Expo Router / React Navigation). The `/src` directory houses all business logic, components, and services organised by feature domain. Each feature folder is self-contained to minimise merge conflicts when multiple developers work in parallel.

2.1 Top-Level Structure

```
balanced-life/
├── app/           # Screen routes (thin layer – imports from src/)
├── src/           # ALL business logic (feature-based)
├── functions/     # Firebase Cloud Functions
├── assets/        # Images, fonts, Lottie animations
├── __tests__/     # Integration / E2E tests
├── firestore.rules # Firestore security rules
├── app.json       # Expo config
├── eas.json       # EAS Build config
├── tailwind.config.js
└── tsconfig.json
```

2.2 Screen Routes (`/app`)

Each file in `/app` becomes a route. Screen files should be thin (20–50 lines) — they import feature components from `/src/features` and compose them.

```
app/
├── _layout.tsx    # Root layout: providers, fonts, auth redirect
├── index.tsx      # Entry redirect (splash logic)
├── (auth)/        # Auth group (no tab bar)
│   ├── _layout.tsx # Stack navigator
│   ├── welcome.tsx # Welcome carousel
│   ├── register.tsx # Account creation
│   ├── login.tsx   # Sign in
│   └── forgot-password.tsx # Password reset
├── (onboarding)/  # Onboarding group (no tab bar)
│   ├── _layout.tsx # Stack navigator
│   ├── assessment-intro.tsx # "Let's find your balance"
│   ├── assessment.tsx # 25-question assessment
│   ├── score-result.tsx # Balance Score reveal + animation
│   └── balance-wheel-intro.tsx # First radar chart view
├── (tabs)/        # Main app (bottom tab bar)
│   ├── _layout.tsx # Tab navigator (Home, Progress, Wheel, Profile)
│   ├── index.tsx   # Dashboard (Home tab)
│   ├── progress.tsx # Progress charts
│   ├── wheel.tsx   # Balance Wheel tab
│   └── profile.tsx  # Settings
├── check-in/      # Modal stack
│   ├── index.tsx   # Daily check-in form
│   └── insight.tsx  # Post check-in insight
├── domain/[id].tsx # Dynamic domain detail
└── journal/       # Journal feature screens
```

└─ budget/	# Budget tracker screen
└─ todos/	# To-do list screen
└─ activity/	# Activity log screen
└─ friends/	# Friend interactions screen

2.3 Business Logic (/src)

src/	
└─ features/	# Feature modules (self-contained)
└─ auth/	# components/ hooks/ services/ stores/ types/
└─ assessment/	# components/ hooks/ services/ constants/ types/
└─ scoring/	# CORE ENGINE
└─ engine/	# assessmentScorer.ts, emaCalculator.ts,
	# balanceFactor.ts, balanceScore.ts,
	# checkInScorer.ts, rollingAverage.ts
└─ check-in/	# components/ hooks/ services/ stores/ types/
└─ dashboard/	# components/ hooks/
└─ balance-wheel/	# components/ hooks/
└─ progress/	# components/ hooks/
└─ domain-detail/	# components/ hooks/
└─ missions/	# components/ engine/ constants/ services/ types/
└─ mood/	# components/ services/ types/
└─ journal/	# components/ constants/ services/ types/
└─ budget/	# components/ services/ types/
└─ todos/	# components/ services/ types/
└─ activity/	# components/ services/ types/
└─ friends/	# components/ services/ types/
└─ notifications/	# hooks/ services/
└─ profile/	# components/ services/ types/
└─ shared/	# Cross-feature reusables
└─ components/	# Button, Card, Input, ScoreCircle, EmptyState...
└─ hooks/	# useFirestore, useNetworkStatus
└─ utils/	# dateUtils, scoreUtils, validators, formatters
└─ types/	# domain.types, navigation.types
└─ config/	# App-wide constants
└─ firebase.ts	# Firebase initialisation
└─ domains.ts	# 5 domain definitions, colours, icons
└─ scoring.ts	# EMA alpha, weights, thresholds
└─ theme.ts	# Colour tokens, spacing, typography
└─ providers/	# AuthProvider, QueryProvider, ThemeProvider

2.4 Firebase Cloud Functions (/functions)

functions/	
└─ src/	
└─ index.ts	# Function exports
└─ scoring/recalculateScores.ts	# Firestore trigger on check-in write
└─ missions/generateWeeklyMissions.ts	# Scheduled: Sunday evening
└─ notifications/sendDailyReminder.ts	# Scheduled: user's preferred time

3. Navigation Structure

The app has 4 navigation groups, with conditional routing based on auth and onboarding state:

3.1 Auth Routing Logic

- Not logged in → Redirect to (auth)/welcome
- Logged in, no assessment → Redirect to (onboarding)/assessment-intro
- Logged in, assessment done → Redirect to (tabs) Dashboard

3.2 Bottom Tab Bar

Tab	Icon	Label	Route
Home	house	Home	(tabs)/index
Progress	chart-line	Progress	(tabs)/progress
Check In (FAB)	plus (elevated)	Check In	Opens check-in/ as modal
Balance	target/radar	Balance	(tabs)/wheel
Profile	user	Profile	(tabs)/profile

The center Check In button is a Floating Action Button (FAB) elevated above the tab bar, making daily check-ins the most prominent action.

4. Firebase Data Model (Firestore)

All user data is stored under `users/{userId}/`. Each subcollection is only readable/writable by the authenticated user (enforced via Firestore security rules).

4.1 Users Collection

```
users/{userId}
├── displayName: string
├── email: string
├── photoURL: string | null
├── createdAt: Timestamp
├── hasCompletedAssessment: boolean
├── notificationToken: string | null
├── notificationPrefs: { enabled, checkInTime, weeklyReport }
├── timezone: string
└── streakData: { currentStreak, longestStreak, lastCheckInDate }
```

4.2 Assessments

```
users/{userId}/assessments/{assessmentId}
├── createdAt: Timestamp
├── answers: { q1: number, q2: number, ..., q25: number }
├── domainScores: { spirituality, family, work, health, financial }
├── balanceScore: number
├── balanceFactor: number
└── isInitial: boolean
```

4.3 Daily Check-Ins

Document ID = date string (YYYY-MM-DD), which enforces one check-in per day.

```
users/{userId}/checkIns/{YYYY-MM-DD}
├── createdAt: Timestamp
├── answers: { spirituality, family, work, health, financial } // 0|1|2 each
├── totalScore: number
└── note: string | null
```

4.4 Daily Score Snapshots

```
users/{userId}/scores/{YYYY-MM-DD}
├── date: Timestamp
├── domainScores: { spirituality, family, work, health, financial }
├── balanceScore: number
├── wellnessLevel: number
├── balanceFactor: number
└── streakCount: number
```

4.5 Weekly Missions

```

users/{userId}/missions/{weekId}           // e.g. "2026-W12"
├─ weekStart / weekEnd: Timestamp
├─ missions: [ { id, domain, title, description, isCompleted, completedAt } ]
└─ completionRate: number

```

4.6 Other Subcollections

Collection	Document Fields
journal/{entryId}	createdAt, title, body, domain, promptUsed, mood
moods/{YYYY-MM-DD}	createdAt, mood (emoji), intensity (1-5), note
expenses/{expenseId}	createdAt, amount, category, description, date
todos/{todoId}	createdAt, title, isCompleted, dueDate, priority, domain
activities/{activityId}	createdAt, type, duration (mins), description, date
friendInteractions/{id}	createdAt, contactName, type, duration, date

4.7 Security Rules

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/{document=**} {
      allow read, write: if request.auth != null
                           && request.auth.uid == userId;
    }
  }
}

```

5. Library-to-Feature Mapping

Feature	Libraries Used
Registration / Login	firebase/auth, React Hook Form, Zod, expo-auth-session
Onboarding Assessment	React Hook Form, react-native-reanimated, gesture-handler
Balance Score Dashboard	react-native-gifted-charts, Zustand, TanStack Query
Radar / Spider Chart	react-native-gifted-charts, react-native-reanimated
Daily Check-Ins	React Hook Form, Zustand, expo-haptics
Weekly Missions	Firebase Cloud Functions, TanStack Query
Progress History	react-native-gifted-charts (LineChart), TanStack Query
Mood Tracker	Custom emoji picker, react-native-gifted-charts
Journal	TextInput (MVP), TanStack Query
Budget Tracker	React Hook Form, gifted-charts (PieChart)
To-Do List	FlatList + Swipeable (gesture handler), TanStack Query
Streak Counter	date-fns, react-native-reanimated
Push Notifications	expo-notifications, Firebase Cloud Functions
Scoring Engine	Pure TypeScript + date-fns (no library needed)
Offline Support	AsyncStorage, Firestore offline persistence (built-in)

6. Implementation Order (6 Phases)

Each phase builds on the previous one. Files within the same phase can be built in parallel by different team members.

Phase 1: Foundation (Sprint 1, Week 1–2)

Goal: Working navigation skeleton, auth, and Firebase connection.

- Project setup: Expo init, TypeScript, ESLint, Prettier, NativeWind, babel/metro config
- Firebase config (firebase.ts) + domain definitions (domains.ts) + theme tokens (theme.ts)
- Shared types: domain.types.ts, navigation.types.ts
- Shared components: Button, Card, Input, LoadingSpinner
- Providers: AuthProvider, QueryProvider
- Root layout (app/_layout.tsx) with providers, font loading, auth redirect
- Auth feature: authService, authStore, useAuth hook, AuthForm components
- Auth screens: welcome, register, login, forgot-password
- Tab navigator shell with placeholder screens
- Firestore security rules
- CI/CD pipeline (GitLab CI: lint + test)

Deliverable: User can install app, create account, log in, see empty dashboard shell.

Phase 2: Assessment + Scoring Engine (Sprint 1–2, Week 3–4)

Goal: Complete onboarding assessment with working scoring.

- Scoring engine (pure TS): assessmentScorer.ts + unit tests
- Scoring engine: balanceFactor.ts + balanceScore.ts + unit tests
- All 25 assessment questions + domain mappings (questions.ts)
- Assessment components: QuestionCard, AnswerOption, ProgressBar, DomainTransition
- Assessment hooks: useAssessment (state, navigation, submission)
- Assessment service: save results to Firestore
- Onboarding screens: assessment-intro, assessment, score-result, balance-wheel-intro
- Shared components: ScoreCircle, AnimatedNumber
- Balance wheel: RadarChart component

Deliverable: User completes 25-question quiz, sees animated score reveal, views radar chart.

Phase 3: Daily Engagement (Sprint 2–3, Week 5–6)

Goal: Daily check-in loop, score evolution via EMA, dashboard.

- Scoring engine: checkInScorer.ts, rollingAverage.ts, emaCalculator.ts + tests
- Scoring engine: scoreBlender.ts (assessment-to-daily transition)
- Check-in components: CheckInCard, CheckInButton, StreakDisplay
- Check-in hooks: useCheckIn, useStreak
- Check-in screens: daily form + post check-in insight

- Dashboard components: ScoreCard, DomainSummary, CheckInPrompt, InsightCard, GreetingHeader
- Insight engine: insightGenerator.ts + templates.ts
- Mission generator: missionGenerator.ts + missionPool.ts
- Cloud Function: recalculateScores (Firestore trigger)

Deliverable: User checks in daily, scores update via EMA, streak tracks, insights shown.

Phase 4: Progress + Domain Tracking (Sprint 3–4, Week 7–8)

Goal: Progress charts, domain details, mood, journal, missions UI.

- Progress feature: ScoreLineChart, DomainProgressCard, TimeRangeSelector
- Domain detail: DomainScoreHeader, DomainTrendChart, SuggestionCard
- Mood tracker: MoodSelector (emoji picker), MoodHistory
- Journal feature: JournalEditor, JournalEntryCard, PromptSuggestion + 30 prompts
- Missions UI: MissionCard, MissionList + Cloud Function for weekly generation

Deliverable: Full progress tracking, journal, mood, and weekly missions working.

Phase 5: Secondary Features (Sprint 5, Week 9–10)

Goal: All remaining trackers and notifications.

- Budget tracker: BudgetSummary, ExpenseEntry, ExpenseList + budget screen
- To-do list: TodoItem, TodoList + todos screen
- Activity logger: ActivityEntry, ActivityHistory + activity screen
- Friend logger: FriendInteraction, InteractionHistory + friends screen
- Notifications: useNotifications hook, notificationService + Cloud Function
- Profile/settings: ProfileHeader, SettingsItem, NotificationPrefs
- Shared: ConfettiOverlay, EmptyState (for all empty screens)

Deliverable: All 15 features functional. App is feature-complete.

Phase 6: Polish & Delivery (Sprint 6, Week 11–12)

Goal: Quality, performance, testing, documentation, showcase prep.

- Unit test coverage: 90%+ for scoring engine
- Integration tests: onboarding flow, check-in flow
- Usability testing with 5–10 real users
- Performance profiling + optimisation
- UI polish: animations, transitions, error boundaries, empty states
- Offline handling: graceful degradation when no network
- EAS Build for TestFlight (iOS) and Internal Testing (Android)
- Documentation: README, architecture doc, handover guide

Deliverable: Production-ready build, tested, documented, ready for Showcase.

7. Team Parallelisation Strategy

The feature-based architecture minimises merge conflicts. Each developer owns one feature folder at a time.

7.1 Role Assignments (4 Devs)

Dev	Focus Area	Owns These Feature Folders
A (Tech Lead)	Backend + Scoring	scoring/, Cloud Functions, firebase config, CI/CD, notifications/
B (Full-Stack)	Core Features	assessment/, check-in/, domain-detail/, profile/
C (Frontend)	Design System + Viz	shared/components/, dashboard/, balance-wheel/, progress/
D (Feature Dev)	Secondary Features	journal/, mood/, missions/, budget/, todos/, activity/, friends/

7.2 Conflict-Prone Files

These files are shared across features. Coordinate carefully:

- app/_layout.tsx — Only one person should own the root layout
- src/config/domains.ts — Domain definitions used everywhere; lock early, change rarely
- src/shared/components/ — Use a "claim before modifying" rule
- package.json — One person adds dependencies to avoid conflicts

7.3 Git Branching Strategy

- main — Always deployable, protected branch
- develop — Integration branch for merging features
- feature/[name] — One branch per feature (e.g. feature/assessment, feature/check-in)
- Merge requests require 1 approval before merging to develop

8. Getting Started (First Commands)

```
# 1. Create the Expo project
npx create-expo-app balanced-life --template expo-template-blank-typescript
cd balanced-life

# 2. Install core dependencies
npx expo install react-native-reanimated react-native-gesture-handler
npx expo install react-native-screens react-native-safe-area-context
npx expo install @react-navigation/native @react-navigation/native-stack
npx expo install @react-navigation/bottom-tabs
npm install zustand @tanstack/react-query
npm install react-hook-form zod @hookform/resolvers
npm install firebase
npm install react-native-gifted-charts react-native-svg expo-linear-gradient
```

```
npm install nativewind tailwindcss
npm install date-fns
npx expo install expo-notifications expo-haptics expo-secure-store
npx expo install @react-native-async-storage/async-storage

# 3. Install dev dependencies
npm install -D @types/react eslint prettier
npm install -D @testing-library/react-native jest

# 4. Start the dev server
npx expo start

# 5. Scan QR code with Expo Go app on your phone
```